Universidad Autónoma de Nuevo León

Facultad de Ingeniería Mecánica y Eléctrica

Componentes de Lógica de Negocios

Producto Integrador de Aprendizaje

VetCare 24

Sistema de gestión para clínica veterinaria 24/7

Reporte final: BRD + arquitectura + evidencias + conclusiones

Integrante	Matrícula	Carrera
David Alejandro Vázquez Fernández	2110193	ITS
Sergio Humberto Román Puente	2099682	ITS
Edelyn Naomi Carrera García	2098207	ITS

Catedrático: Ing. Miguel Ángel Gómez Marroquín · Grupo 002 · Hora N1-N3 · Periodo Agosto-Diciembre 2026

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León, noviembre de 2026

Repositorio	https://github.com/sergiohroman/PIA_EQ4_CLN
Sistema desplegado / Swagger UI	https://<servicio>.onrender.com/docs (ver README del repositorio)

1. Resumen y alcance

VetCare 24 es una clínica veterinaria que ofrece consultas, vacunas, cirugías, estética, estudios y hospitalización. Durante las Unidades 2 y 3 el equipo levantó el análisis (BRD, historias de usuario, BPMN y glosario) y diseñó la arquitectura. Este PIA convierte ese trabajo en una aplicación funcional, verificable y desplegable, construida con **DDD, Specification, State Machine y una API REST documentada** (FastAPI + SQLAlchemy + PostgreSQL), contenerizada con Docker Compose y con pipeline CI/CD en GitHub Actions.

Problema de negocio. Duplicación de expedientes y clientes, agenda sin duración por servicio, emergencias sin prioridad formal, expedientes en papel, recordatorios manuales y falta de control de la capacidad de hospitalización (10 jaulas).

Alcance implementado: agenda con duración por servicio y espacios libres (N-001), emergencias con prioridad (N-007), expedientes con unicidad por celular, fusión de duplicados y múltiples mascotas (N-002, N-004), notas y recetas con firma inmutable (N-005), recordatorios por Domain Events (N-003), hospitalización con límite de 10 jaulas y signos vitales (N-006) y reportes de productividad, ocupación y vacunas atrasadas (N-008).
Fuera de alcance (documentado): facturación, inventario de medicamentos, estética y estudios como servicios, autenticación/roles, envío real a WhatsApp (se entrega adaptador con *outbox*) y gráficos de productividad.

2. Análisis del negocio

2.1 Necesidades, restricciones y criterios de éxito (BRD)

Código	Necesidad / restricción / criterio
N-001	Organizar la agenda por tipo de servicio (consulta, vacuna, cirugía) mostrando espacios libres en tiempo real
N-002	Historia médica en un solo lugar; varias mascotas por dueño con un dato único (celular)
N-003	Avisos automáticos de vacunas por WhatsApp 3 días antes y el mismo día
N-004	Detectar clientes existentes y fusionar archivos repetidos
N-005	Notas, recetas y alergias digitales con firma del veterinario
N-006	Pacientes internados con signos vitales cada 4 a 6 h y máximo 10 jaulas
N-007	Urgencias médicas con prioridad sobre las citas programadas
N-008	Reportes: vacunas pendientes, consultas por doctor y ocupación del internado
BR-001 / BR-002 / BR-003	Protección de datos personales · máximo 10 mascotas internadas · entrega en 10 semanas
CE-001 a CE-004	Cero duplicados · historial en menos de 10 s · 8 de cada 10 pacientes regresan a refuerzos · cero papeles

2.2 Historias de usuario (15) con criterios de aceptación Gherkin

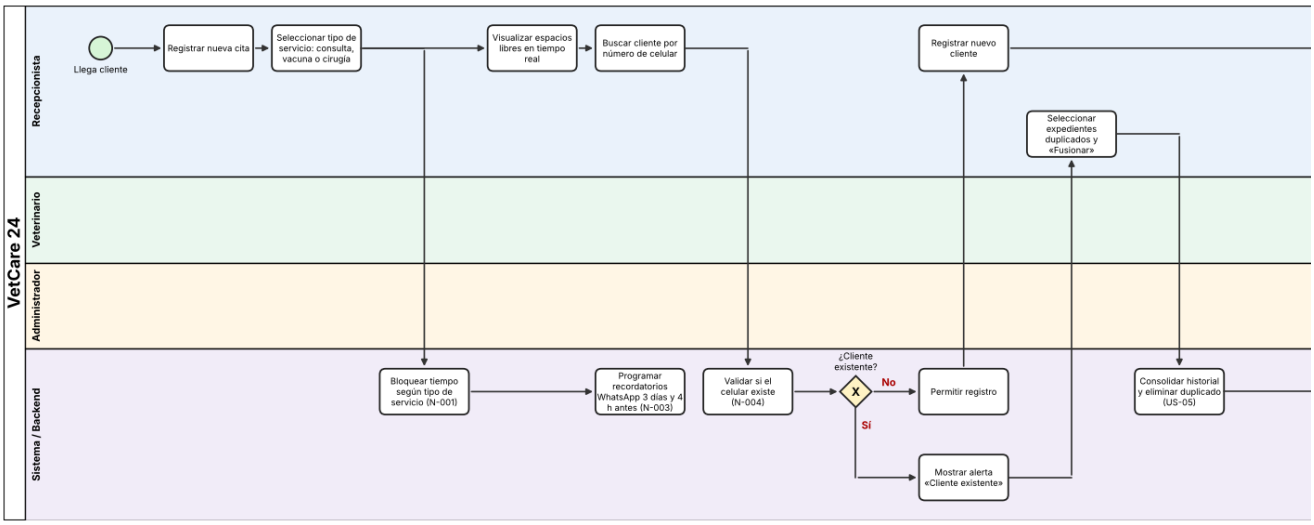
El archivo `docs/analisis/historias_usuario.md` contiene las 15 historias con escenarios Gherkin (28 escenarios). Todas implementadas; US-09, US-10 y US-14 con alcance parcial (ver trazabilidad).

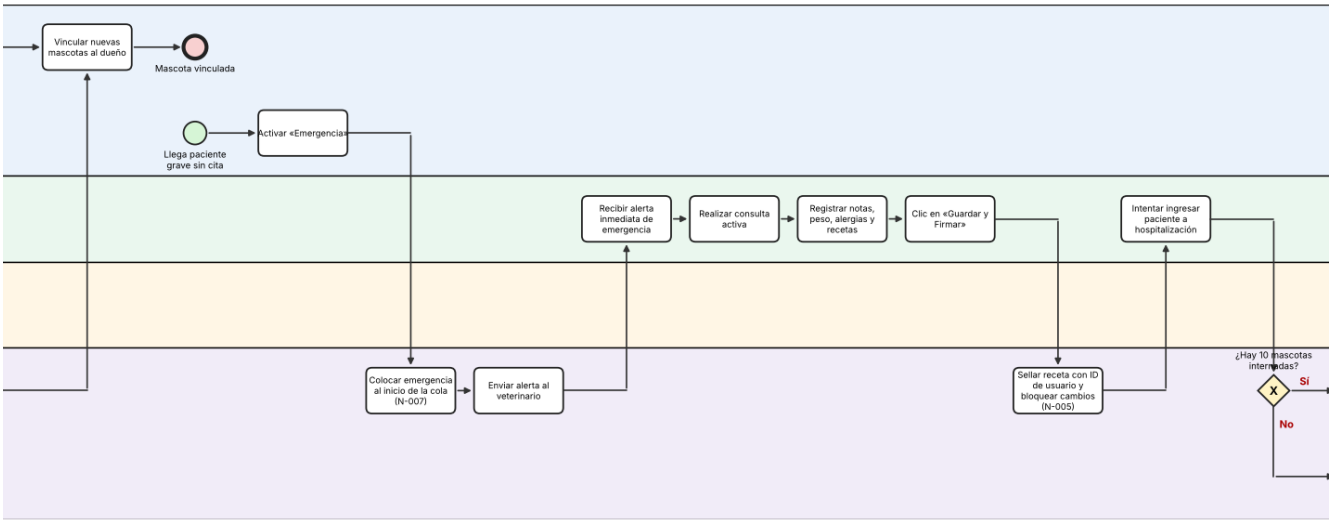
Módulo	Historias	Necesidad
Agenda y citas	US-01 duración automática · US-02 espacios libres · US-03 emergencia	N-001, N-007
Expedientes y pacientes	US-04 alerta de celular existente · US-05 fusión · US-06 varias mascotas por dueño · US-07 notas y recetas · US-08 firma digital	N-002, N-004, N-005
Recordatorios	US-09 aviso 3 días antes · US-10 aviso 4 h antes	N-003
Hospitalización	US-11 signos vitales cada 4 a 6 h · US-12 límite de 10 jaulas	N-006

Módulo	Historias	Necesidad
Reportes	US-13 vacunas atrasadas · US-14 productividad por veterinario · US-15 ocupación de internado	N-008

2.3 Proceso de negocio (BPMN)

El BPMN 2.0 (`docs/analisis/bpmn/vetcare24.bpmn`) modela el pool VetCare 24 con cuatro carriles: Recepcionista, Veterinario, Administrador y Sistema/Backend. Incluye las decisiones «¿Cliente existente?» y «¿Hay 10 mascotas internadas?». Cada tarea se asigna a un endpoint o regla en la sección «BPMN → código» de `docs/trazabilidad.md`.





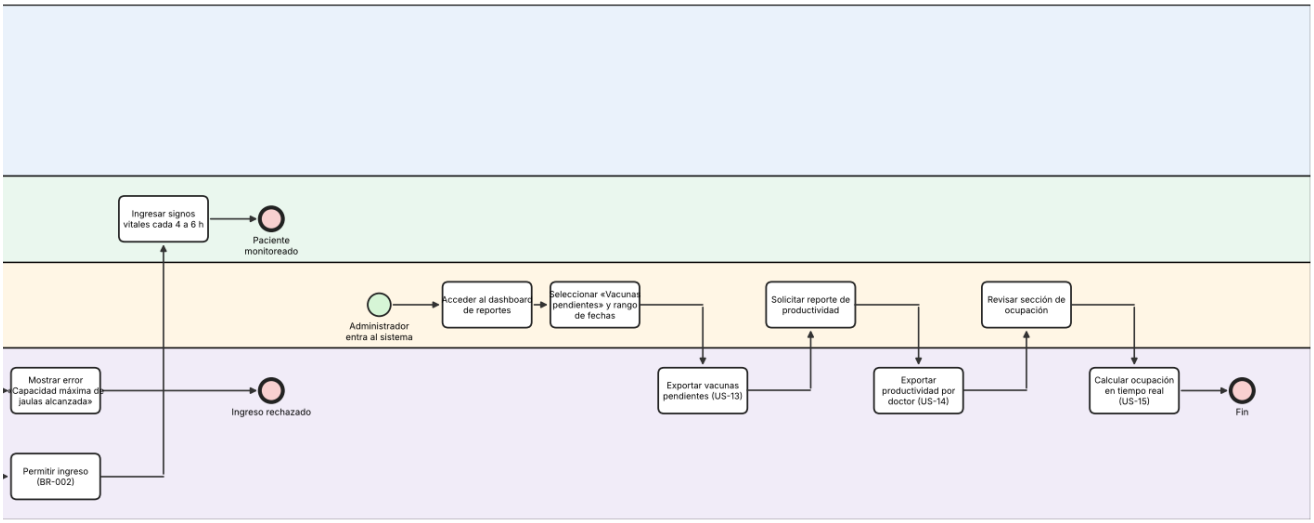


Figura 1. BPMN de VetCare 24 (se muestra en tres tramos; el archivo completo está en el repositorio).

2.4 Glosario de dominio

El glosario (`docs/analisis/glosario.md`) define el lenguaje ubicuo y lo vincula con las clases del código: Propietario, Mascota (Paciente), Expediente Clínico, Cita, Agenda, Hospitalización, Nota Clínica, Receta Médica, Firma Digital, Emergencia, Capacidad Máxima, Vacuna atrasada, entre otros. Se unificó «celular» frente a «teléfono» en todos los artefactos.

3. Arquitectura y diseño

3.1 Modelo de dominio (DDD)

Se eligió **Domain Model** frente a Transaction Script y Table Module porque las reglas son invariantes que deben cumplirse siempre (ADR-001). Hay tres agregados, cada uno con su repositorio:

Agregado (raíz)	Contenido	Invariante protegida
ExpedienteClinico	Propietario, Mascotas, NotasClinicas	Celular único por propietario (N-004); mascotas agrupadas bajo un dueño (N-002); nota firmada inmutable (N-005)
Agenda	Citas del día (estado por State Machine)	Sin traslape por veterinario salvo emergencia (N-001, N-007)
Hospitalizacion	10 Jaulas, SignosVitales	Nunca más de 10 jaulas ocupadas (BR-002); versionado optimista

Value Objects: Celular, SignosVitales, TipoServicio, RecetaMedica, FirmaDigital. **Domain Services:** ValidadorDuplicidadClienteService y AsignadorPrioridadEmergenciaService. **Domain Events:** CitaAgendadaEvent, EmergenciaRegistradaEvent, PacienteInternadoEvent, CapacidadJaulasAlcanzadaEvent, CitaCambioEstadoEvent.

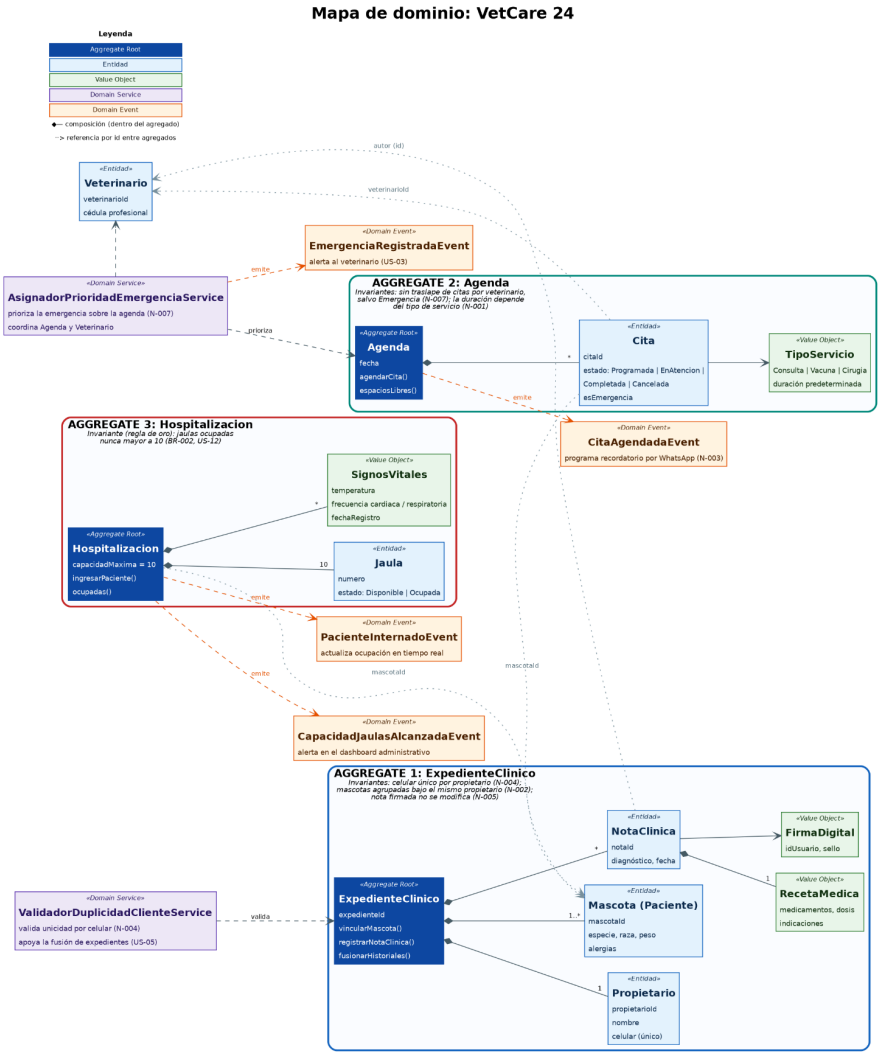


Figura 2. Mapa de dominio (Actividad 3).

3.2 Capas y diagramas UML

Capa	Carpeta	Responsabilidad
Dominio	src/domain	Entidades, VO, agregados, Specifications, State Machine, servicios y eventos, puertos de repositorio. No importa SQLAlchemy ni FastAPI
Aplicación	src/application	Casos de uso, DTOs, Commands/Queries (CQRS básico), UoW (puerto), bus de eventos
Infraestructura	src/infrastructure	SQLAlchemy (ORM, repositorios, UoW), adaptador WhatsApp, base de datos
API	src/api	Controllers REST, esquemas Pydantic, traducción de errores a HTTP, composition root

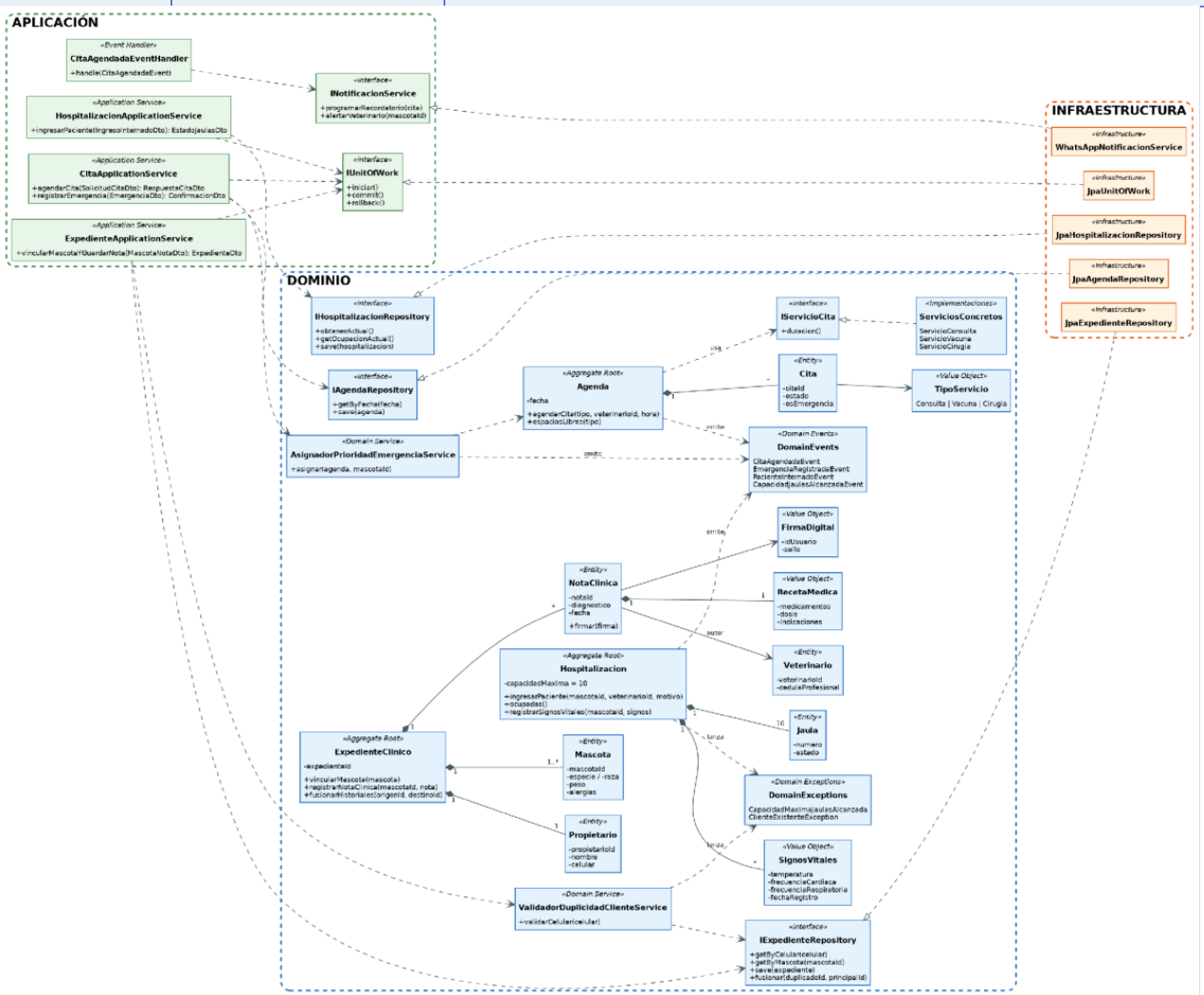


Figura 3. Diagrama de clases por capas (diseño de la Actividad 3). Los diagramas Mermaid que reflejan el código final están en docs/architecture/diagramas.md.

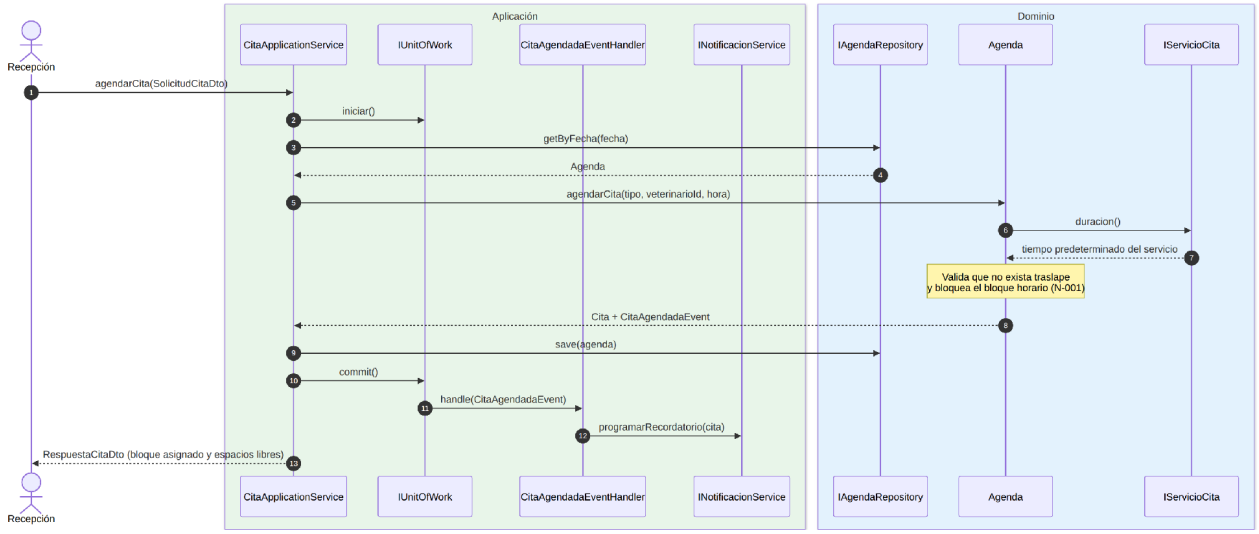


Figura 4. Secuencia CU-01: agendar cita con asignación automática de tiempo (N-001).

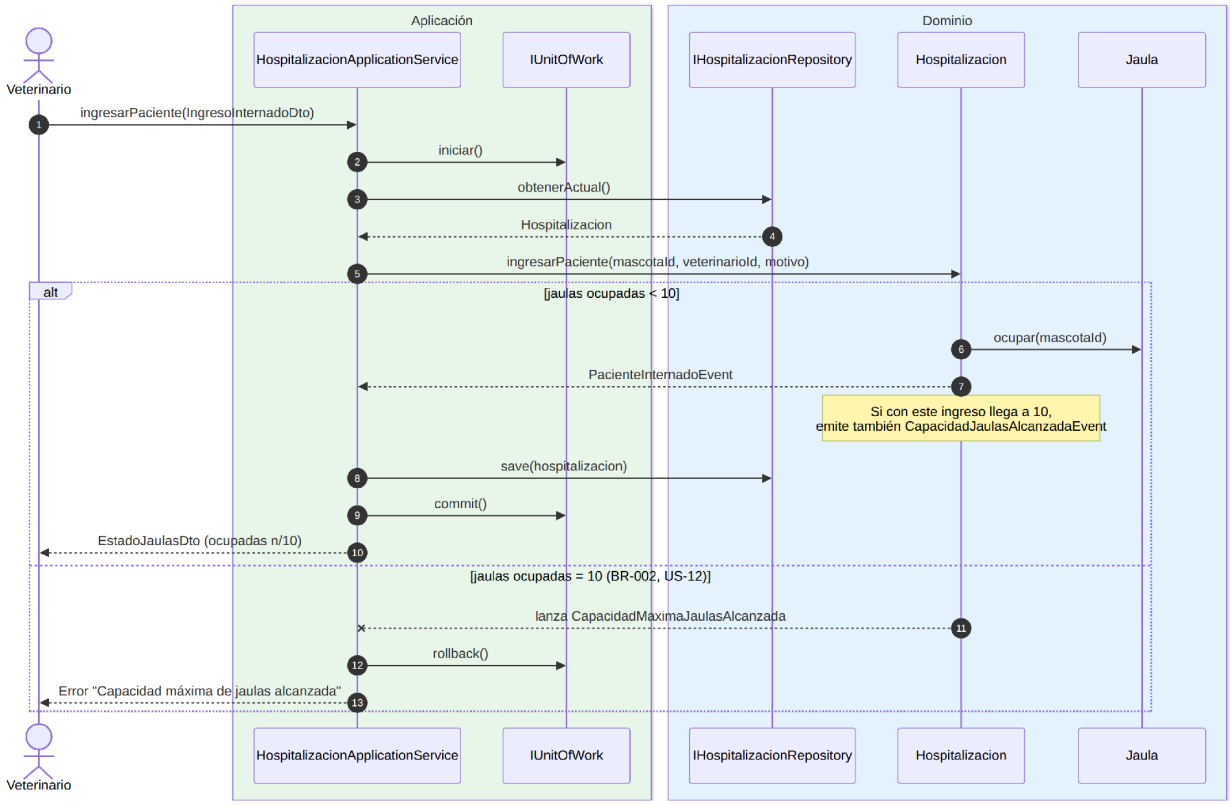


Figura 5. Secuencia CU-04: ingreso a hospitalización, con rechazo y rollback (BR-002).

3.3 Patrones de diseño y justificación

Patrón	Dónde	Problema del negocio que resuelve
Domain Model + Aggregates	domain/expediente.py, agenda.py, hospitalizacion.py	Invariantes críticas: 10 jaulas, firma inmutable, celular único
Specification (AND/OR/NOT)	domain/specification.py, specifications.py	Reglas combinables: «sin traslape O emergencia», «tiene cupo», «vacuna atrasada» (ADR-005)
State Machine	domain/estados.py	Ciclo de vida de Cita y de NotaClinica; rechaza transiciones inválidas (ADR-006)

Patrón	Dónde	Problema del negocio que resuelve
Repository + Unit of Work	domain/repositories.py, infrastructure/	Persistencia desacoplada y una transacción por caso de uso con rollback (ADR-002)
Domain Events	domain/events.py, application/event_bus.py	Recordatorios y alertas sin acoplar la agenda a WhatsApp (ADR-003)
Strategy	domain/servicios.py	Duración por servicio; agregar estética sin modificar Agenda (OCP)
CQRS básico, DTO	application/commands.py, queries.py, dtos.py	Separar escritura (reglas) de lectura (historial, ocupación, reportes)
Optimistic Locking	infrastructure/repositories.py	Dos ingresos simultáneos no pueden exceder 10 jaulas (ADR-004)

3.4 Principios SOLID (evidencia en el código)

Principio	Evidencia
S	CitaApplicationService solo coordina; el envío de mensajes lo hace CitaAgendadaEventHandler mediante INotificacionService
O	Nuevo servicio = nueva clase IServicioCita registrada, sin tocar Agenda; nueva regla = nueva Specification
L	Cualquier IExpedienteRepository / IAgendaRepository es sustituible (SQL o doble de prueba, p. ej. RepoFalso en las pruebas unitarias)
I	Puertos de repositorio pequeños por agregado en lugar de un ClinicaRepository de 30 métodos
D	Los Application Services reciben uow_factory (abstracción); el Contenedor inyecta las implementaciones concretas

3.5 Architecture Decision Records

ADR	Decisión	Alternativas descartadas	Trazabilidad
001	Domain Model como patrón de lógica de negocio	Transaction Script, Table Module	N-002, N-004..N-007, BR-002
002	Un Repository por Aggregate Root y Unit of Work por caso de uso	Repositorio general; uno por tabla; SQL desde el servicio	N-001, N-004, N-006
003	Domain Events para recordatorios y alertas	Llamada directa a WhatsApp; proceso programado	N-003, N-007
004	La regla de las 10 jaulas vive en el agregado Hospitalización, con bloqueo optimista	Validar en el servicio; restricción solo en BD	BR-002, N-006, US-12
005	Patrón Specification con composición AND/OR/NOT	if encadenados; booleanos sueltos	N-001, N-004, N-008
006	State Machine para Cita y NotaClinica	if por estado; Enum sin tabla de transiciones	RN-07, RN-11
007	Python + FastAPI + SQLAlchemy + PostgreSQL + Docker + Render	Java/Spring (JPA), C#/.NET	RNF-03..05

Cada ADR (contexto, decisión, alternativas, consecuencias, trazabilidad) está en `docs/architecture/`. Diferencias respecto al diseño de la Actividad 3 (JPA a SQLAlchemy, incorporación de Specification y State Machine, elementos de la bitácora de IA que no se adoptaron) se documentan en `decisiones_tecnicas.md` §4.

4. Implementación

La API REST (FastAPI) expone los casos de uso con Swagger UI en `/docs`, ReDoc en `/redoc` y el contrato en `/openapi.json` (exportado a `docs/api/openapi.json` por el CI). Las reglas del negocio violadas responden **409**, los datos inválidos **422** y los recursos inexistentes **404**.

Endpoint	Caso de uso / historia
POST /expedientes · GET /expedientes/{celular}	Alta de propietario; 409 si el celular existe (US-04)
POST /expedientes/{celular}/mascotas · .../mascotas/fusion	Vincular mascota (US-06) y fusionar duplicados (US-05)
POST /expedientes/{celular}/mascotas-con-nota	CU-03 en una sola transacción
POST /mascotas/{id}/notas · PUT /notas/{id} · POST /notas/{id}/firma	Notas, recetas y firma digital; 409 si la nota ya está firmada (US-07, US-08)
GET /mascotas/{id}/historial	Historial de la mascota (CE-002)
POST /citas · GET /citas/espacios-libres	CU-01: duración automática, traslape 409 (US-01, US-02)
POST /emergencias · GET /citas/cola	CU-02: emergencia primero en la cola y alerta al veterinario (US-03)
POST /citas/{id}/transiciones	State Machine: aprobar, completar, cancelar
POST /hospitalizacion/ingresos · /altas/{id}	CU-04: 409 «Capacidad máxima de jaulas alcanzada» con 10 ocupadas (US-12)
POST /hospitalizacion/signos-vitales · GET /signos-pendientes · GET /ocupacion	US-11 y US-15
GET /reportes/productividad · GET /reportes/vacunas-pendientes	US-14 y US-13 (JSON o CSV con ?formato=csv)
POST /reglas/operaciones/evaluar · GET /notificaciones	Specifications compuestas del paquete de pruebas; outbox de WhatsApp (US-09, US-10)
POST/GET /veterinarios · GET /health	Catálogo de veterinarios; verificación de salud para el despliegue

Specification. Ejemplo real: la regla de la agenda se expresa como `CitaSinTraslape(citas) | CitaEsEmergencia()`, y el reporte de la US-13 como `CitaEsVacuna() & CitaVencida(ahora) & CitaConEstado(PROGRAMADA, CANCELADA)`.

State Machine. Cita: PROGRAMADA → EN_ATENCION (aprobar) → COMPLETADA (completar); cancelar desde PROGRAMADA o EN_ATENCION. NotaClinica: BORRADOR → FIRMADA (firmar). Una transición inválida lanza TransicionInvalida y la API responde 409.

Persistencia y transacciones. SQLAlchemy 2.0 sobre PostgreSQL (SQLite en memoria para pruebas). Cada caso de uso se ejecuta dentro de un Unit of Work; si el dominio rechaza la operación (por ejemplo la jaula 11) se hace rollback y la ocupación permanece en 10/10. Los eventos de dominio se publican después del commit.

5. Calidad, automatización y despliegue

5.1 Pruebas y cobertura

La suite combina pruebas **unitarias** del dominio (agenda, expediente, hospitalización, Value Objects, Specification, State Machine, reglas) y de **integración** sobre la API y la base de datos (citas, expedientes, hospitalización, reportes, persistencia). Resultado de la ejecución local del 05/11/2026 en Python 3.14 (`python -m pytest`), previa a la incorporación de la US-13:

Métrica	Resultado
Pruebas ejecutadas	103 aprobadas, 0 fallidas

Métrica	Resultado
Cobertura total	98.19 % (1,710 sentencias, 31 sin cubrir); umbral exigido 80 %, configurado en pytest.ini para fallar si no se alcanza
Dominio	100 % en agenda, hospitalización, estados, eventos, Specification, operaciones; 96–98 % en expediente, value objects
Aplicación e infraestructura	100 % en queries, DTOs, comandos, ORM; 97 % servicios; 98 % repositorios
API	100 % en todos los routers; 97 % main.py

Las nuevas pruebas de la US-13 (`test_us13_*`, unitarias e integración) cubren: vacuna no asistida, cancelada, completada, en atención, aún no vencida, no vacuna, filtro por rango, exportación CSV y rango inválido (422).

Además, `scripts/evaluar_casos_vetcare.py` ejecuta los 120 casos y 20 transiciones del paquete de pruebas VetCare 24 y deja la evidencia en `docs/evidencias/`.

5.2 CI/CD, Docker y variables de entorno

Elemento	Detalle
GitHub Actions (.github/workflows/ci.yml)	Job «pruebas»: instala dependencias, ejecuta pytest (falla con menos de 80 % de cobertura), exporta OpenAPI y sube coverage.xml. Job «docker»: pruebas contra PostgreSQL con Docker Compose y verificación de /health
Docker Compose	Servicios db (PostgreSQL 16), app (FastAPI) y tests (perfil «test»), con healthcheck de la base de datos
12-Factor	Configuración por variables de entorno (.env.example sin secretos): POSTGRES_*, DATABASE_URL, APP_PORT, TEST_DATABASE_URL; la plataforma inyecta DATABASE_URL y PORT
Despliegue	Blueprint render.yaml (servicio web con Docker + PostgreSQL gestionado), healthCheckPath /health, Swagger UI activo en /docs

6. Trazabilidad análisis - arquitectura - código - pruebas

La matriz completa está en `docs/trazabilidad.md`. Resumen:

Necesidad	Historia	Componentes principales	Endpoint	Decisión
N-001	US-01, 02	Agenda, Cita, IServicioCita	POST /citas, GET /citas/espacios-libres	ADR-001, 002
N-003	US-09, 10	CitaAgendadaEvent, WhatsAppNotificacionService	GET /notificaciones	ADR-003
N-007	US-03	AsignadorPrioridadEmergenciaService	POST /emergencias, GET /citas/cola	ADR-001, 003
N-002, N-004	US-04, 05, 06	ExpedienteClinico, ValidadorDuplicidadClienteService	POST /expedientes, .../fusion	ADR-001, 002
N-005	US-07, 08	NotaClinica, RecetaMedica, FirmaDigital	POST /notas/{id}/firma	ADR-001, 006
N-006, BR-002	US-11, 12	Hospitalizacion, Jaula, HospitalizacionTieneCupo	POST /hospitalizacion/ingresos	ADR-004, 002
N-008	US-13, 14, 15	vacuna_atrasada, Queries (CQRS)	GET /reportes/*, GET /hospitalizacion/ocupacion	ADR-005, 001

7. Uso de IA (bitácora)

La IA se usó como herramienta de consulta para proponer estructuras, borradores de ADR y revisiones SOLID; todas las decisiones de arquitectura y negocio fueron revisadas y validadas por el equipo. La bitácora de la Actividad 3 menciona propuestas que no se adoptaron (Event Sourcing, Facturación, TriageLevel, C#); el equipo las descartó por alcance y plazo y lo documentó en `decisiones_tecnicas.md` §4 y ADR-007.

8. Limitaciones conocidas y trabajo futuro

Limitación	Mitigación / siguiente paso
Sin autenticación ni roles	Agregar OAuth2/JWT y roles (Recepción, Veterinario, Administración); hoy la firma usa un veterinario_id válido
Outbox de notificaciones en memoria, sin proveedor real	Integrar WhatsApp Business API con reintentos y registro de fallos (ADR-003)
create_all en lugar de migraciones	Adoptar Alembic
Fusión solo de mascotas, no de propietarios duplicados	Extender ValidadorDuplicidadClienteService a fusión de expedientes de personas
Vacuna atrasada = cita de vacuna vencida y no aplicada; el BRD no define esquemas por especie	Catálogo de esquemas de vacunación y refuerzos por especie
Productividad solo en tabla JSON	Gráficos en un frontend

9. Conclusiones

El proyecto convierte los artefactos de análisis y diseño de las Unidades 2 y 3 en un sistema funcional donde cada regla importante del negocio vive en un solo lugar del dominio y puede rastrearse hasta una necesidad del BRD, una historia de usuario, un endpoint y una prueba. Modelar con DDD permitió probar las invariantes más críticas (10 jaulas, celular único, receta firmada, prioridad de emergencia) sin base de datos, y los patrones Specification y State Machine volvieron explícitas reglas que de otro modo habrían quedado como condicionales dispersos. La separación en capas y el uso de repositorios y Unit of Work hicieron posible cambiar de JPA a SQLAlchemy sin tocar el dominio. La automatización (pruebas con umbral de 80 %, CI, Docker Compose y variables de entorno) deja el entorno reproducible. Como aprendizaje, la trazabilidad temprana evitó inconsistencias entre BRD, diagramas y código, y las decisiones se documentaron como ADR para poder defenderlas.

Anexos en el repositorio: BRD, historias de usuario, BPMN, glosario, ADR-001 a ADR-007, decisiones técnicas, diagramas, evidencias y README con la guía de demostración.